Syntax Directed Translation

Date / /
Page No.

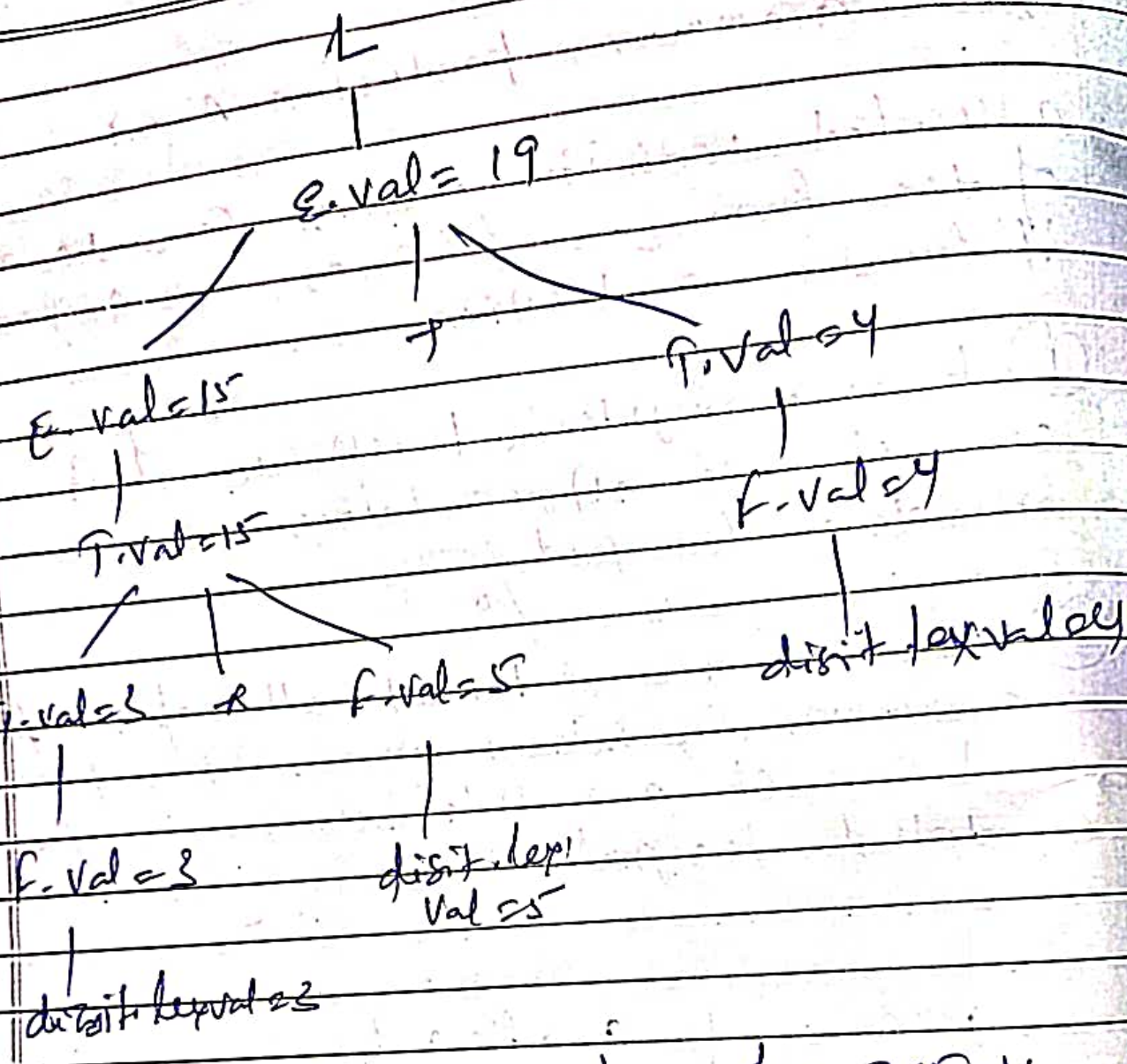
Each grammar production $A \to \alpha$ has associated with it a set of semantic rules of the form $b = f(c_1, c_2, \ldots c_k)$ where 'f' function and either

(1) b is a Synthesized attribute of A and $c_1 \ldots c_k$ are attrs belonging to the grammar symbols of the prod / or

(2) b is an inherited attr of one of the grammar symbols on the R.H.S of the prod and $c_1 \ldots c_k$ are attrs belonging to the grammar symbols of the prod.

Prod	Semantic rules
Augmented grammar → L → E	Print (E.val)
E → E + T	E.val = E.val + T.val
E → T	E.val = T.val
T → T * F	T.val = T.val * F.val
T → F	T.val = F.val
F → (E)	F.val = E.val
F → digit	F.val = digit.lexval

Syntax directed translation def!n



Annotated parse tree for 3*5+4

Synthesized

Sp:

E.val → f.val

E.val

↑

f.val

Inherited

E.val = f.val

E.val

↓

f.val

Synthesized

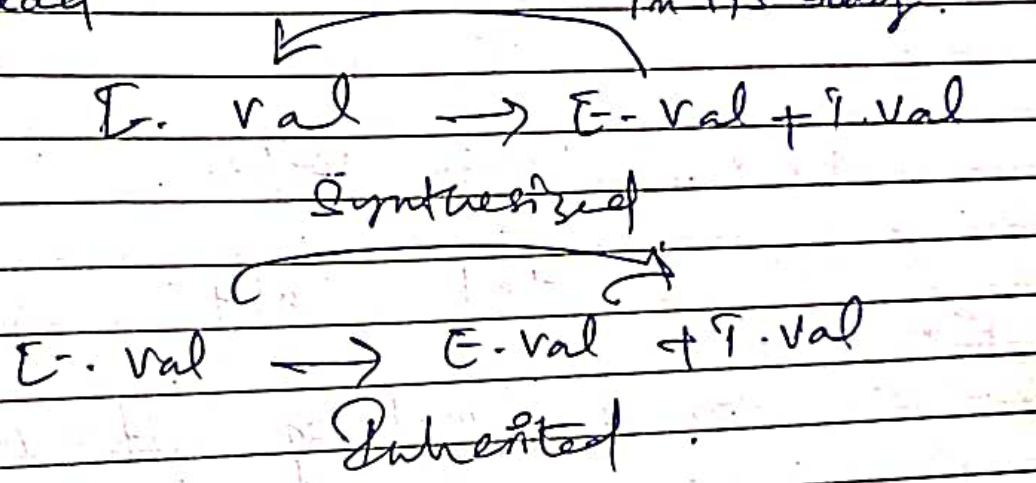
→ In Parse tree node value is det by the attr at child nodes

→ prod must have Non terminal its head

Inherited

→ Value is determined by the attr value at Parent node.

→ production must have N.T as a symbol in its body.



Intermediate code generation

3 types

1. Postfix Notation
2. Syntax Tree
3. Three address code — Quadruples
   Triples
   Indirect Triples

1) Postfix Notation

Infix	Prefix	Postfix
a+b	+ab	ab+

Why?

→ for better implementation Postfix Notation is used.

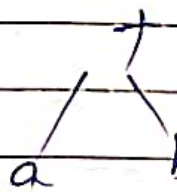
→ No associativity

→ No precedence relations

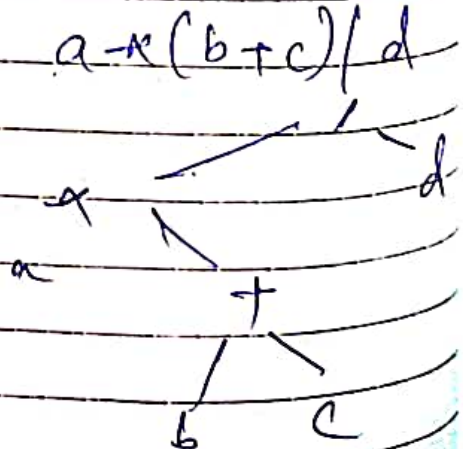
Ex:- (a+b)*c ab+c*

2) Syntax Tree

a+b



a*(b+c)/d



Infix : left root right

Prefix : root left right

Postfix : left right root

Infix : $a * b + c / d$

Prefix : $/ * a + b c d$

Postfix : $a b c + * d /$

③ Three Address Code

Ex 1 : $(a+b) * (a+b+c)$

$t_1 = a+b$

$t_2 = a+b$

$t_3 = t_2+c$

$t_4 = t_1 * t_3$

Ex 2 : $a + a * (b-c) + d * (b-c)$

TAC → $t_1 = b-c$

$t_2 = a * t_1$

$t_3 = a + t_2$

$t_4 = d * t_1$

$t_5 = t_3 + t_4$

one result

Eg: a = -b * c + d

$t_1 = -b$ — (1)

$t_2 = c + d$ — (2)

$t_3 = t_1 * t_2$ — (3)

$a = t_3$ — (4)

↓

Result :

	operator	opr1	opr2	Result
(1)	Uminus	b	-	t_1
(2)	+	c	d	t_2
(3)	*	t_1	t_2	t_3
(4)	=	t_2	-	a

Quadruple Example

Eg:- a = -b * c + d

TAC :

$t_1 = -b$

$t_2 = c + d$

$t_3 = t_1 * t_2$

$a = t_3$

	operator	opr1	opr2
(1)	-	b	-
(2)	+	c	d
(3)	*	1	2
(4)	=	3	-

Triple Example

Quadruples (4 fields)

one operator

two operands

one result

Ex: $a = -b * c + d$

$t_1 = -b$ — (1)

$t_2 = c + d$ — (2)

$t_3 = t_1 * t_2$ — (3)

$a = t_3$ — (4)

Result:

	operator	opr1	opr2	Result
(1)	Uminus	b	-	t_1
(2)	+	c	d	t_2
(3)	*	t_1	t_2	t_3
(4)	=	t_2	-	a

Quadruple Example

Triples (3 fields)

1 operator

2 operands

Ex:- $a = -b * c + d$

TAC:

$t_1 = -b$

$t_2 = c + d$

$t_3 = t_1 * t_2$

$a = t_3$

	operator	opr1	opr2
(1)	-	b	-
(2)	+	c	d
(3)	*	(1)	(2)
(4)	=	3	-

Triple example

Translation of Assignment Statement

In SDT, assignment stmt is mainly deal with expressions.

Ex: $S \rightarrow id = E$

$E \rightarrow E_1 + E_2$

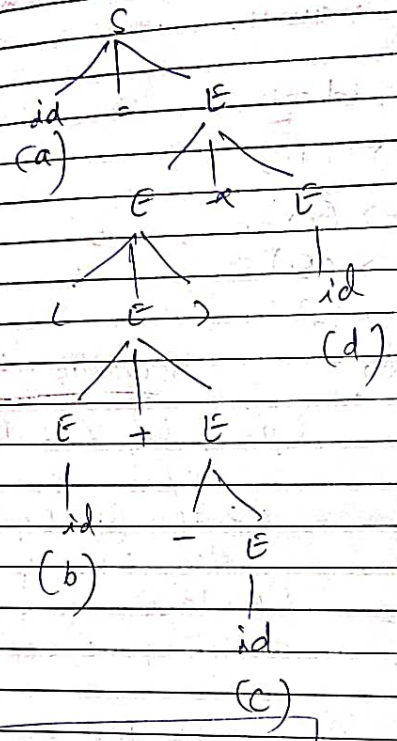
$E \rightarrow E_1 * E_2$, $E \rightarrow -E$

$E \rightarrow (E_1)$

$E \rightarrow id$

Production	Semantic Action
$S \rightarrow id = E$	{ p = lookup (id.Name) if p ≠ NULL then Emit (P = E.Place) else error }
$E \rightarrow E_1 + E_2$	{ E.Place = Newtemp; Emit (E.Place = E_1.Place + E_2.Place) }
$E \rightarrow E_1 * E_2$	{ E.Place = Newtemp; Emit (E.Place = E_1.Place * E_2.Place) }
$E \rightarrow -E$	{ E.Place = Newtemp; Emit (E.Place = Uminus E_1.Place) }
$E \rightarrow (E_1)$	{ E.Place = E_1.Place }
$E \rightarrow id$	{ p = lookup (id.Name); if p ≠ NULL then E.Place = P else error }

Ex: a = (b + c) * d




```
t1 = Uminus c
t2 = b + t1
t3 = t2 * d
a = t3
```

```
100: if a<b goto 103
101: t = 0
102: goto 104
103: t = 1
```

Boolean Expressions

Date / /
Page No.

for 2 purposes use Boolean Expr's.

1. used for computing logical values
2. used as conditional expressions

1. E → E OR E
2. E → E AND E
3. E → NOT E
4. E → (E)
5. E → id relop id → relational operator <, >, =
6. E → true = 1
7. E → false = 0

prod Rule	Semantic Action

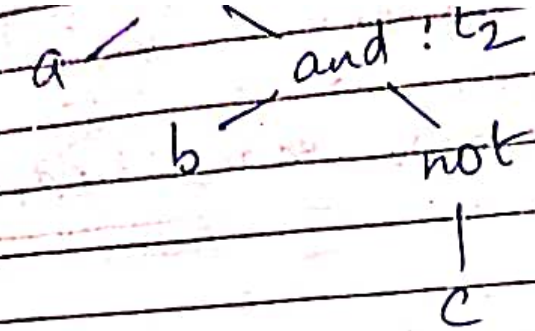
for 2 purposes use Boolean Expr.

(1) used for computing logical values

(2) used as conditional expressions

(1) $E \rightarrow E$ OR E
(2) $E \rightarrow E$ AND E
(3) $E \rightarrow$ NOT E
(4) $E \rightarrow (E)$
(5) $E \rightarrow$ id relop id → relational operator $<, >, =$
(6) $E \rightarrow$ true $= 1$
(7) $E \rightarrow$ false $= 0$

prod Rule	Semantic Action
(1) $E \rightarrow E_1$ OR E_2	{ E.place = Newtemp(); Emit(E.place = E_1.place OR E_2.place) }
(2) $E \rightarrow$ id$_1$ relop id$_2$	{ E.place = Newtemp(); emit(if id$_1$.place relop.op id$_2$.place goto nextstat+3); Emit(E.place = 0); Emit(goto nextstat+2); Emit(E.place = 1); }



$t_1 = \text{not } c$

$t_2 = b \text{ and } t_1$

$t_3 = a \text{ or } t_2$

If a<b then 1 else 0

100 : if a<b goto 103

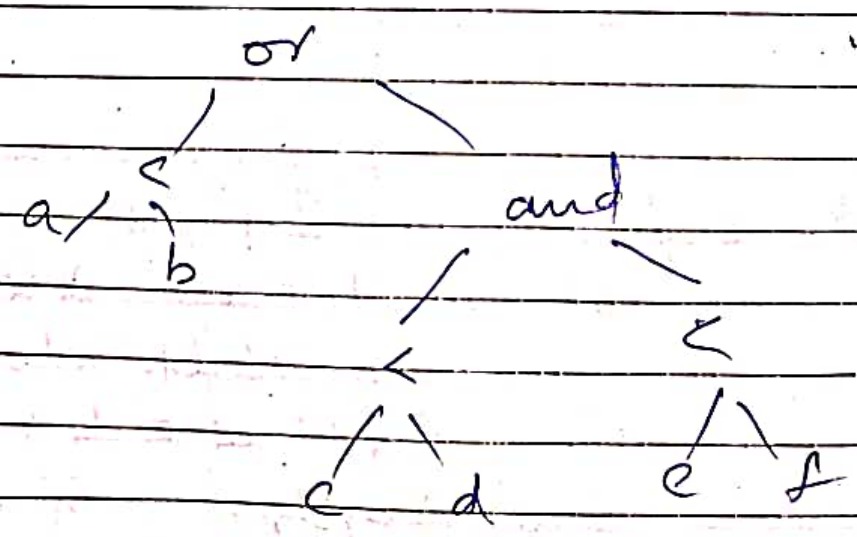
101 : $t_1 = 0$

102 : goto 104

103 : $t_1 = 1$

a<b or c<d and e<f

t_1 (t_2) (t_3)



Date
Page No.

~~100: $t_1 = 0$~~

1st

100: if $a < b$ goto 103
101: $t_1 = 0$
102: goto 104
103: $t_1 = 1$

2nd

104: if $c < d$ goto 107
105: $t_2 = 0$
106: goto 108
107: $t_2 = 1$

3rd

108: if $e < f$ goto 111
109: $t_2 = 0$
~~110: goto 112~~
~~110:~~
110: goto 112
111: $t_3 = 1$

112: $t_4 = t_2$ and t_3

113: $t_5 = t_1$ or t_4

Date:
Page No.

If else.

3-address code for If then else

If a<b then x = y+z else p = q+r

(1) If a<b goto 3 (true)
(2) goto 6 (false)
(3) $t_1 = y + z$
(4) $x = t_1$
(5) goto
(6) $t_2 = q + r$
(7) $p = t_2$
(8) goto

while (a<b) do x = y+z.

(1) If (a<b) goto 3 (true)
(2) goto ___ (false)
(3) $t_1 = y + z$
(4) $x = t_1$
(5) goto (1)
(6) ___

If else.

3- address code for If then else

If $a<b$ then $x=y+z$ else $p=q+r$

(1) If $a<b$ goto 3 (true)

(2) goto 6. (false)

(3) $t_1 = y+z$

(4) $x = t_1$

(5) goto

(6) $t_2 = q+r$

(7) $p = t_2$

(8) goto

while (a<b) do x=y+z.

(1) If (a<b) goto 3 (true)

(2) goto ___ (false)

(3) $t_1 = y+z$

(4) $x = t_1$

(5) goto (1)

(6)

for loop

Date / /
Page No.

(i = i + 1)

```
for (i=1; i<=20; i++)
    a = y + 3;
```

① i = 1 ;

② If (i <= 20) goto 7 { True }.

③ goto --- { false }

④ $t_1 = i + 1$

⑤ $i = t_1$

⑥ goto ②

⑦ $t_2 = y + 2$

⑧ $a = t_2$

⑨ goto 4

#

Arrays

* One dimensional Array A.

A[1]	...	A[i]		A[n]

$Base_A$ = Address of starting loc element

width = ~~de loc~~ in bytes.

low = Index of first array element

low of A[1] = 1 low of A[i] = i

* loc of A[i] = $BASE_A + (i - low) * width$

A[1]	A[2]	A[3]	A[4]
10	14	18	22	26

Base = 10

width = 4

low = 1 & 4

st end

endloc - st loc

loc of A[4] = $10 + (4-1) * 4$

$= 10 + 3 * 4$

$= 22$

Date / /
Page No.

$Base_A + i \times width - low \times width$

$= (Base_A - low \times width) + i \times width$

$= (address_A - width) + i \times width$

($address_A - width$ = t_2; $i \times width$ = t_1)

$\Rightarrow A[i] = t_2[t_1]$

Ex:- Convert the following into 3 address code.

$a[i] = 10$; width = 4 bytes.

① $t_1 = i \times width = i \times 4$

② $t_2 = (addr_A - width)$ = to

③ $t_2[t_1] = 10$

(t_1 = i, t_2 = $a[i]$)

2-dimensional Array

$A[i, j] = base_A + (i - low_1) n_2 + (j - low_2) \times width$

n_2 = no. of columns.

$base_A + i$

2-dimensional Array

$A[i, j] = base_A + ((i - low_1) \times n_2 + (j - low_2)) \times width$

n_2 = No. of columns.

$= base_A + i \times n_2 \times width - low_1 \times n_2 \times width + j \times width - low_2 \times width$

$= \underbrace{(i \times n_2 + j) \times width}_{t_1} + base_A - \underbrace{(low_1 \times n_2 + low_2) \times width}_{t_2}$

$A[i, j] = t_2[t_1]$

$base_A$ = Starting addr of A

low_1, low_2 = Ind of 1st row, 1st col

n_2 = no. of elements in column

width = width of each array elmt

i, j = index of i^{th} row j^{th} column

Date / /
Page No.

Eg:- add = 0 ✓
i = 1
j = 1

add = add + a[i, j] + b[j, i]

a and b array size 20 × 20

4 bytes / Word = width

Assume : low$_1$ = 1 low$_2$ = 1

for A[i, j] = t$_2$[t$_1$]

(i × 20 + j) × 4 + address$_A$ - (1 × 20 + 1) × 4

(i × 20 + j) × 4 + (address$_A$ - 164)
t$_1$ t$_2$

for B[j, i] = t$_4$[t$_3$]

(j × 20 + i) × 4 + address$_B$ - (1 × 20 + 1) × 4

= (j × 20 + i) × 4 + address$_B$ - 164
t$_3$ t$_4$

#

(1) add = 0
(2) i = 1
(3) j = 1
(4) $t_1 = i \times 20$
(5) $t_1 = t_1 + j$
(6) $t_1 = t_1 \times 4$
(7) $t_2 = addr_A - 164$
(8) $t_5 = t_2[t_1]$
(9) $t_2 = j \times 20$
(10) $t_2 = t_2 + i$
(11) $t_2 = t_2 \times 4$
(12) $t_4 = addr_B - 164$
(13) $t_6 = t_4[t_2]$
(14) $t_7 = t_5 * t_6$
(15) $t_8 = add + t_7$
(16) add = t_8

for
add = add +
a[i, j] + b[i, j]

Ex. 1 Switch

Date __/__/__
Page No.

```
Switch (ch)
{
case 1: c = a+b;
        break;
case 2: c = a-b
        break;
}
```

Sol

```
If ch=1 goto L1
If ch=2 goto L2
L1:
    T1 = a+b
    c = T1
    goto last
L2:
    T1 = a-b
    c = T2
    goto last
last:
```

```
c=0
do
{
if (a<5) then
  a++;
else
  a--;
  c++;
}while (c<=5)
```

Sol 1. c = 0
2. if (a<b) goto (4)
3. goto (7)
4. T1 = x+1
5. x = T1
6. goto (9)
7. T2 = x-1
8. x = T2
9. T3 = c+1
10. c = T3
11. if (c<5) goto (2)
12.

4- Unit
Symbol Table & Error Handler

Symbol Table Entries

```
int x;
char y[] = "Hello world";
```

Name	Type	Size	Dimension	Line of Declaration	Line of Usage	Address
x	int	2	0	—	—	
y	char	12	1	—	—	

Data Structures for Symbol Tables

(i) Lists
(ii) Self organising Lists
(iii) Search Trees
(iv) Hash Tables

c_1 : diagnostic : Illegal Parantheses

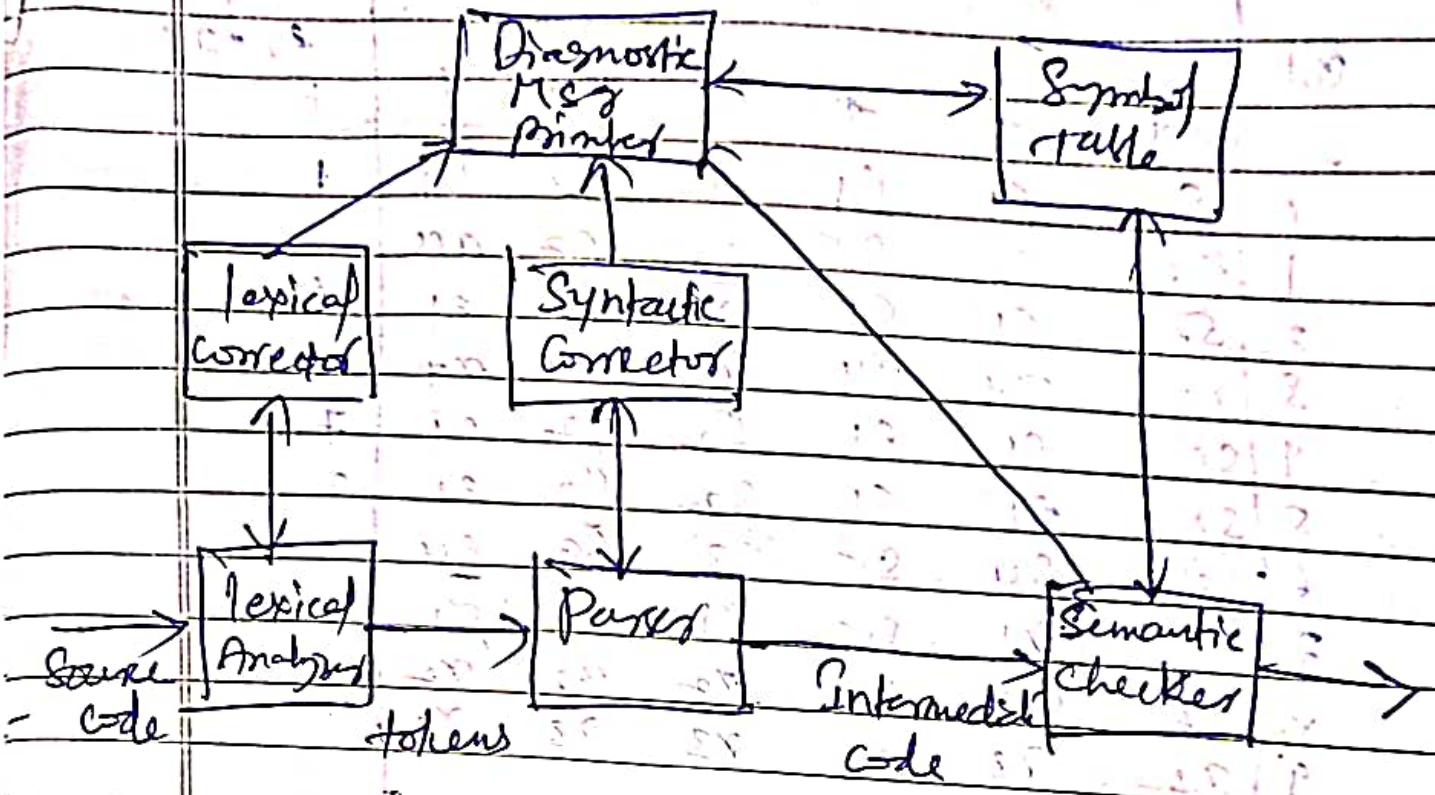
c_2 : diagnostic : Missing operator

c_3 : Diagnostic : Illegal right Parantheses

c_4 : Diagnostic : Missing expression

Date / /
Page No.

Plan of error Detector & Corrector



operator Precedence Matrix with error Entries

$E \to EAE \mid (E) \mid -E \mid id$

$A \to + \mid - \mid * \mid /$

id + (id * id) + id $

	+	*	(	)	id	$
+	>	<	<	>	<	>
*	>	>	<	>	<	>
(	<	<	<	=	<	e_1
)	>	>	e_2	>	e_2	>
id	>	>	e_2	>	e_2	>
$	<	<	<	e_3	<	e_4

LR Parsing : Error routines

State	action						goto
	id	+	*	(	)	$	E
0	s3	e1	e1	s2	e2	e1	1
1	(e3)	s4	s5	(e3)	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	(e3)	s4	s5	(e3)	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

$E \to E + E$
$E \to E * E$
$E \to id$

e1 : D : Missing operand

e2 : D : Unbalanced right parenthesis

e3 : D : Missing operator

e4 : D : Missing Right Parenthesis

Stack	input	
0	id +) $	
0 id 3	+) $	
0 E 1	+) $	
0 E 1 + 4	) $	
0 E 1 + 4	$	Unbalanced)
0 E 1 + 4 id	$	Push id
0 E 1 + 4 id 3	$	
0 E 1 + 4 E 7	$	

State / Symbol	id	+	*	(	)	$
E	E→TE'	e1	e1	E→TE'	e1	e1
E'	E'→ε	E'→+TE'	e1	E'→ε	E'→ε	E'→ε
T	T→FT'	e1	e1	T→FT'	e1	e1
T'	T'→ε	T'→ε	T'→*FT'	T'→ε	T'→ε	T'→ε
F	F→id	e1	e1	F→(E)	e1	e1
id	Pop					
+		Pop				
*			Pop			
(				Pop		
)	e2	e2	e2	e2	Pop	e2
$	e3	e3	e3	e3	e3	~~Accept~~

e1 : Missing operand
e2 : Missing right parenthesis
e3 : Unexpected.

E → TE'
E' → +TE' / ε
T → FT'
T' → *FT' / ε
F → (E) / id.

Stack	i/p
$E	id +) $
$ E' T	id +) $
$ E' T' F	id +) $
$ E' T' id	id +) $
$ E' T'	+) $
$ E'	+) $
$ E' T +	+) $
$ E' T	) $
$ E' T' F	) $
$ E'	$

Pop id / Mismatch found
Unexpected))

Basic Block : It is a sequence of consecutive statements in which the flow control enters at the beginning and leaves at the end without branching or halt.

Algo to Convert 3-add code to Basic Block

1. Determine the leader from 3-add-code
   a. 1st stmt of 3 add code is a leader
   b. Target statement of Unconditional/Conditional goto is a leader
   c. Stmt immediately following Un/Conditional goto is a "leader"
2. The Basic Block starts at one "leader" and ends just before the next "leader"

Flow graph : Fg is directed graph where nodes are BB's and edges are used to add the flow of info from one block to another.

Loop in flow graph : loop is a collection of nodes in a fg such that there is a path from any node to any other node within that loop.

Code

(1) prod = 0 ⇒ leader (1)

(2) i = 1

(3) $t_1 = i * 4$ ⇒ leader (2)

(4) $t_2 = addr[A] - 4$

(5) $t_2 = t_2[t_1]$

(6) $t_4 = addr(B) - 4$

(7) $t_5 = t_4[t_3]$

(8) $t_6 = t_2 * t_5$

(9) $t_7 = prod + t_6$

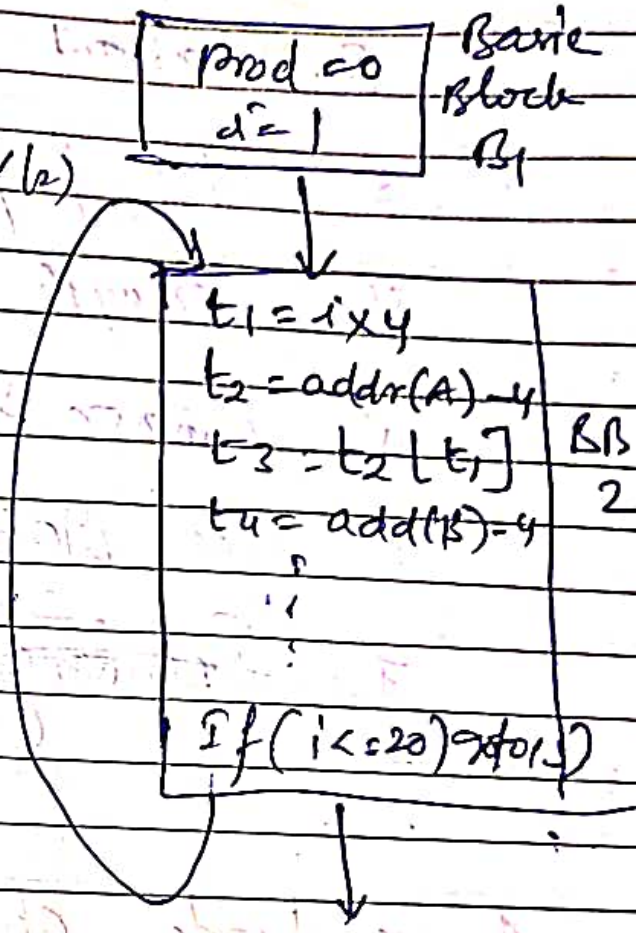
(10) prod = t_7

(11) $t_8 = i + 1$

(12) $i = t_8$

(13) if ($i <= 20$) goto (3)

(14) _____ leader (3)



Code optimization techniques

Date
Page No.

1. Dead code elimination
2. Constant folding } Compile time evaluation
3. Copy propagation }
4. Strength Reduction
5. Common Sub expr elimination
6. Code Motion
7. ~~Loop~~ Unrolling

1. Dead code elimination

Before elimination

```
c = a*b
a = b          → dead code
d = a*b+4
```

After

```
c = a*b
d = a*b+4
```

2. Constant folding

Ex: a = b op c

b, c → constants

pi = 3.14
radius = 10
Area of circle = pi*r*r
= 3.14*10*10
= 314

$y = 10$

3. Copy propagation

Before	After
$c = a * b$	$c = a * b$
$x = a$	$x = a$
till	till
$d = x * b + 4$	$d = a * b + 4$

4. Common Sub expr Elimination

Before	After
$c = a * b$	$c = a * b$
$x = a$	$x = a$
till	till
$d = a * b + 4$	$d = c + 4$

Common Sub exp elimination

Strength Reduction

Code before opt	Code After opt
$s_1 = 4 * i$	$s_1 = 4 * i$
$s_2 = a[s_1]$	$s_2 = a[s_1]$
$s_3 = 4 * j$	$s_3 = 4 * j$
~~$s_4 = 4 * i$~~	$s_5 = n$
$s_5 = n$	$s_6 = b[s_1] + s_5$
$s_6 = b[s_4] + s_5$	

Strength Reduction

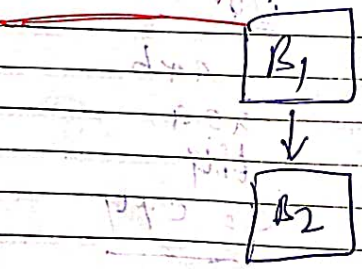
Before

$B = A \times 2$

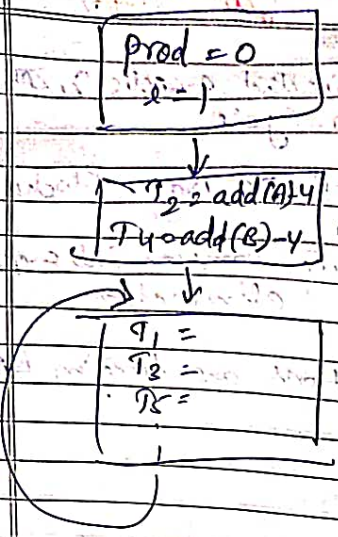
After

$B = A + A$

Code Motion



Date
Page No.



add(A) & add(B) will not change
These are static
These can be computed at compile time only.

Compile time	Runtime
(1) at compile time	at Runtime
(2) Prevents error before execution	(2) Can not prevent error before exec
(3) Contains Syntax & semantic errors ; }	(3) Contains error such as div by zero, square root of -ve Number

DAG (Directed A cyclic Graph)

DAG also called Directed A cyclic graph that contains NO cycle

DAG is used to optimize Basic Blocks

used to eliminate common sub exp elimination.

How the calculations are done in BB.

Algo

1. leaf node represent Id, Name & Const
2. New node is created on hyothesis such a node doesn't exists
3. Assignment of the a=y must not perform

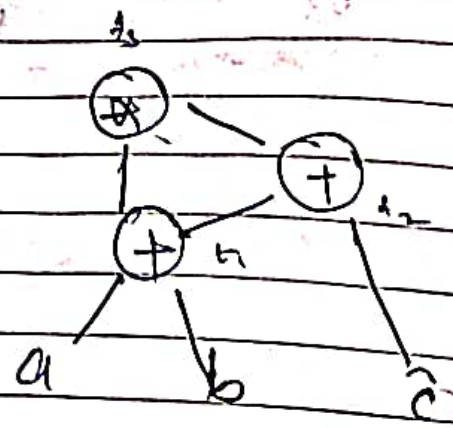
Exp: $(a+b) * (a+b+c)$

$t_1 = a+b$

$t_2 = t_1 + c$

$t_3 = t_1 * t_2$

DAG



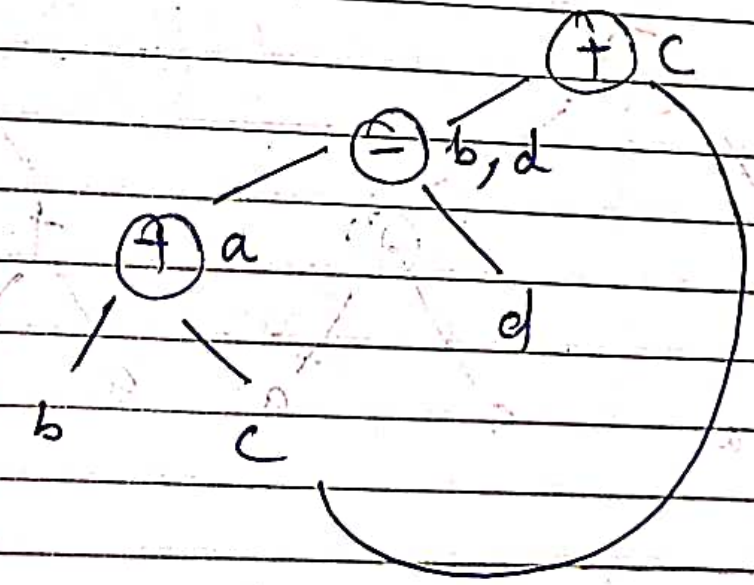
Eg2:

$a = b + c$

$b = a - d$

$c = b + c$

$d = a - d$



Date
Page No.

Ex3:- $X = ((a+a)+(a+a)+((a+a)+(a+a)))$

t_1 t_2 t_3 t_4 t_5 t_6

$t_1 = a+a$

$t_2 = a+a$

$t_3 = t_1+t_2$

$t_4 = a+a$

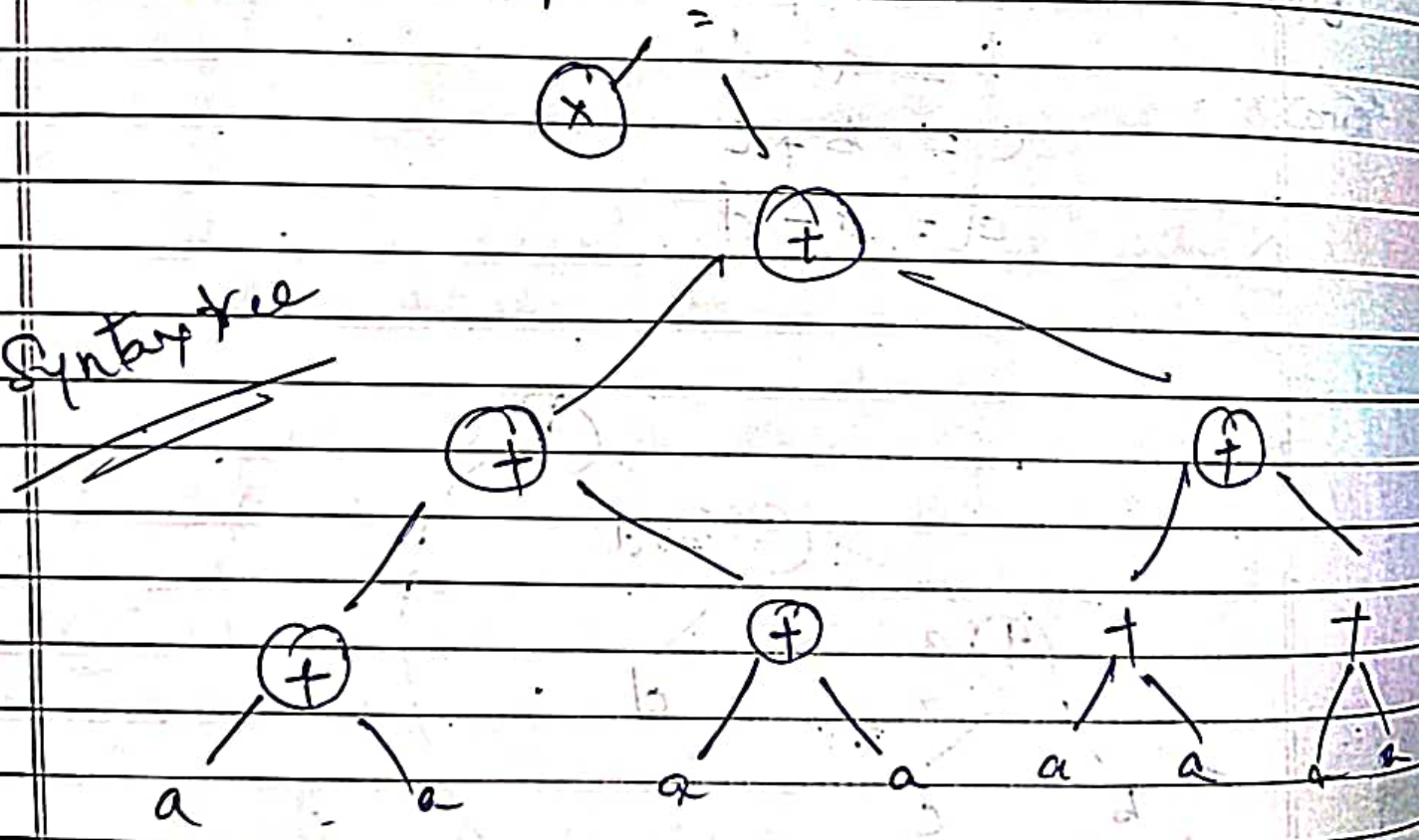
$t_5 = a+a$

$t_6 = t_4+t_5$

$t_7 = t_3+t_6$

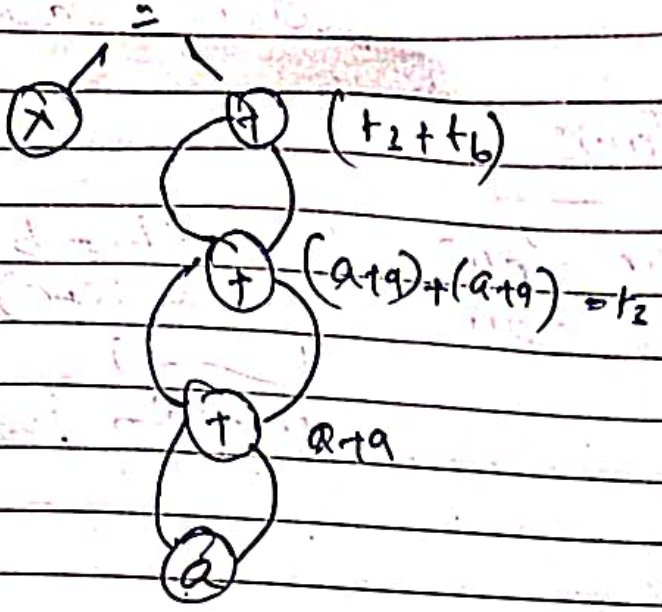
$X = t_7$

Syntax tree

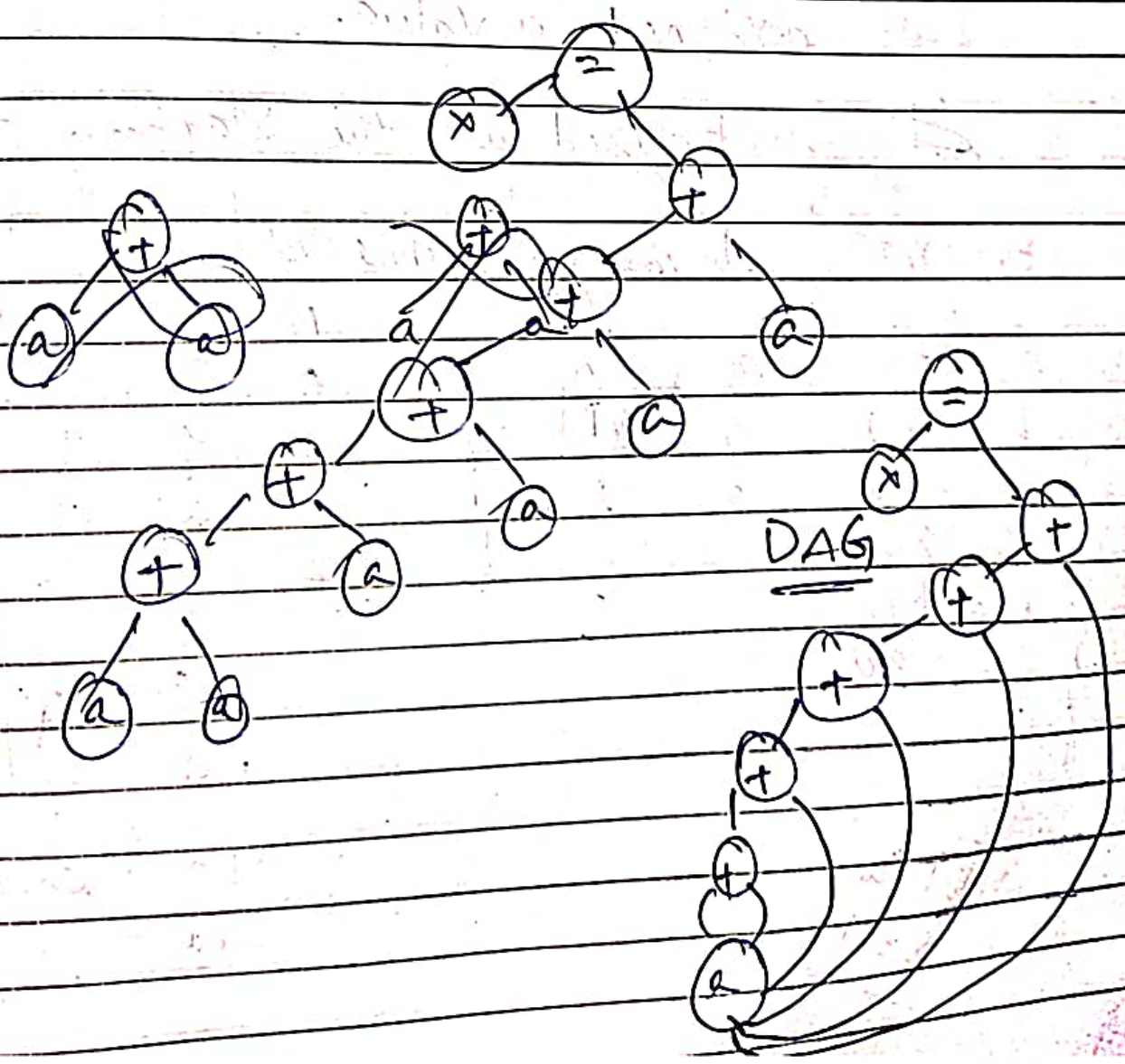


Date / /
Page No.

DAG



Eg 4 $x = a+a+a+a+a+a$



used to improve the optⁿ

Value Number

It is a No. associated with each Var used within the block. It uniquely identifies the place in the BB where the Var was last assigned a value

initialized at the starting of BB

Expression	value	value No	value	val No
i) a = x * y	a	0	x	0
ii) b = x * y	y	1	y	1
iii) c = a + i	a	2	a	2
iv) x = y			b	2
v) d = b + i				
vi) a = a * d				

x	0
y	1
a	2
b	2
i	3
c	4
d	4

c = a + i
d = b + i
} where a = b

if (a > 1) then a = 1

Global Data Flow Analysis

Date / / Page No.

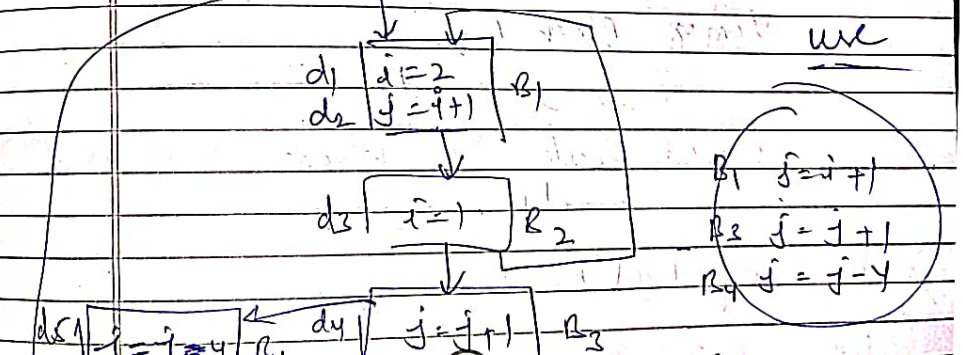
Obj: is to compute ud (use definition chaining)

use: Meaning of use is that id A is used at point p

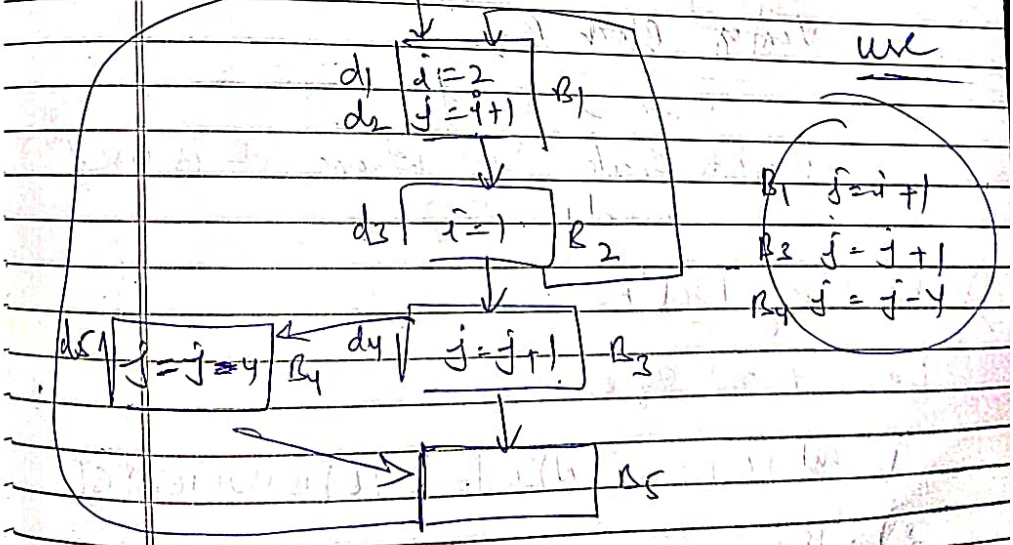
Def'n: an assignment to A

Gen[B]: is the set of generated def'ns those definitions within Block B that reach at the end of the block

[KILL][B]: is the set of def'ns outside of B that defines id's that also have def'ns within B.



defns within B.



use

B1 j=i+1
B2 j=j+1
B4 j=j-4

GEN & KILL of

Block	GEN[B]	Bit vector	KILL[B]	B# Vector
B1	{d1, d2}	11000	{d3, d4, d5}	00111
B2	{d3}	00100	{d1}	00000
B3	{d4}	00010	{d2, d5}	01001
B4	{d5}	00001	{d2, d4}	01010
B5	ϕ	00000	ϕ	00000

KILL → Verify Block B1

Var. i, j used

need to check where these two are again used.

∴ Those are d3, d4, d5

Verify Block B2

Var i used

need to check 'i' where it is used in d1

Reaching Def'ns

Data flow eq'ns:

1) out[B] = IN[B] - KILL[B] ∪ GEN[B]

2) IN[B] = ∪ predecessor of p out[P]

Reaching Def'ns

Data flow eq's:

1) $OUT[B] = IN[B] - KILL[B] \cup GEN[B]$

2) $IN[B] = \bigcup_{\text{predecessor of } p} OUT[P]$

Computation of IN and OUT

Date / / Page No.

Block	GEN[B]	IN[B] initial	OUT[B] initial	IN[B] Pass 1	OUT[B] Pass 1
B_1	00000	00000	11000	00100	11000
B_2	00100	00000	00100	11000	01100
B_3	00010	00000	00010	01100	00110
B_4	00001	00000	00001	00110	00101
B_5	00000	00000	00000	00111	00111

$IN[B_1]$ = Union of Predecessor B_1

$IN[B_2] = B_1$

$IN[B_3] = B_2$ & B_5 = 00100 | 00000 = 00100

$IN[B_4] = B_3$

$IN[B_5] = B_3 + B_4$ = 00100

00100 | 00111 = 00111

01100 | 00111 = 01111

11000 | 00111 = 11111

$OUT[B_1] = IN[B_1] - KILL[B_1] \cup GEN[B_1]$

$00100 \wedge \neg KILL[B_1] \cup GEN[B_1]$

$00100 \wedge \neg(00111) \cup GEN[B_1]$

$00100 \wedge 11000 \cup GEN[B_1]$

$00000 \cup 11000$

$= 11000$

Pass 2 Pass 3 & Pass 4 - - -

Both must same to stop the process

Computing Ud Chains

* If a Use of a Var A is preceded in its block by a definition of A, then only the last definition of A in the block prior to this Use reaches the Use. Thus Ud Chain for this Use consists of only this one definition.

* If a Use of A is preceded in its block B by no definition of A then the ud chain for this use consists of all definitions of A in IN[B]

* It is a DS used to store reaching definitions

Block	Variable use	Ud chain
B_1	I	d_1 ✓
B_2	No Var use	— ✓
B_3	J	$\{01111\} = \{d_0, d_2, d_4, d_5\}$
B_4	J	$\{00110\} = \{d_3, d_4\} = \{d_4\}$
B_5	J	$\{00111\} = \{d_2, d_4, d_5\}$

$d_1 = i = 2$ } B_1
$d_2 = j = i+1$ } B_1
$d_3 = i = 1$ } B_2
$d_4 = j = j+1$ } B_3
$d_5 = j = j-4$ } B_4

$00110 = \{d_3, d_4\}$
$00101 = \{d_3, d_5\}$
$00111 = \{d_3, d_4, d_5\}$

Pass 2 — only B_2

B_1	00 01100	11000
B_2	11111	01111
B_3	01111	00110
B_4	00110	00101
B_5	00111	00111

Pass 3

	01111	11000
	11111	01111
→	01111	00110
→	00110	00101
→	00111	00111